

Handbook

All scripts are run from the **project root** (language-quality-and-publication-success/). All input and output paths are relative to that directory.

Prerequisites

Python version

Python **3.10 or later** is required.

Installation

Create and activate a virtual environment, then install all dependencies from `requirements.txt`:

```
python -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows:
.venv\Scripts\activate

pip install -r requirements.txt
```

Note on torch: `requirements.txt` installs the CPU-only build by default. For GPU support, install torch separately before running `pip install -r requirements.txt`:

```
# Replace cu121 with your CUDA version (e.g. cu118, cu124):
pip install torch==2.5.1 --index-url https://download.pytorch.org/whl/cu121
```

Note on vllm: Only required for `qwen_native_like_prediction.py` (GPU cluster only). If you do not need it, you can skip it:

```
pip install -r requirements.txt --exclude vllm
```

Runtime requirements by stage

Stage	CPU/GPU	RAM	Approx. time
1 — Sampling	CPU	4 GB	~5 min
2 — Abstract cleaning	CPU	4 GB	~5 min
3–4 — PDF/JSON conversion	CPU	2 GB	varies
5 — Abstract readability	CPU	4 GB	~2 h (all 50 shards)
5 — Abstract perplexity	GPU recommended	8 GB RAM + 8 GB VRAM	~1 h/shard
6 — Paper readability	CPU	4 GB	varies
6 — Grammar	CPU sufficient	4 GB RAM	varies

Stage	CPU/GPU	RAM	Approx. time
detection			
6 — Native-likeness (Qwen 14B)	GPU required	32 GB RAM + 24 GB VRAM	varies
7 — Validation	CPU	2 GB	< 1 min
8–9 — Preprocessing / splits	CPU	8 GB	~5 min
10 — EDA	CPU	8 GB	~10 min
11 — Model selection	CPU	8 GB	~30–60 min
12 — Model training (per script)	CPU	8 GB	~5–20 min
13 — Ablation (25 models each)	CPU	8 GB	~2–4 h each
14 — Feature importance	CPU	2 GB	< 1 min

Folder Structure

```

language-quality-and-publication-success/
├── thesis/                                # Thesis PDF
├── data/                                  # All datasets (see below)
│   ├── full_arxiv_before_sampling.csv
│   ├── 90k_arxiv_metadata_from_semantic_scholar.csv
│   ├── 90k_arxiv_citation_prediction_full.csv
│   ├── 90k_arxiv_doi_prediction_full.csv
│   ├── 200-paper-benchmark_validation_of_linguistic_features.csv
│   ├── 90k_arxiv_citation_prediction_splits/
│   │   ├── 90k_arxiv_citation_prediction_train.csv
│   │   ├── 90k_arxiv_citation_prediction_val.csv
│   │   └── 90k_arxiv_citation_prediction_test.csv
│   ├── 90k_arxiv_doi_prediction_splits/
│   │   ├── 90k_arxiv_doi_prediction_train.csv
│   │   ├── 90k_arxiv_doi_prediction_val.csv
│   │   └── 90k_arxiv_doi_prediction_test.csv
│   └── 200-paper-benchmark/                # 200 papers used for zero-shot prompting
and feature validation
│   ├── dev/
│   │   ├── native-like/                   # 50 native-like papers (json, txt,
grammar, qwen outputs)
│   │   └── non-native-like/               # 50 non-native-like papers
│   └── test/
│       ├── native-like/                   # 50 native-like papers (2 batches)
│       └── non-native-like/               # 50 non-native-like papers (2 batches)
├── code/                                  # All source code (see below)
│   ├── 01_data_collection/
│   ├── 02_feature_extraction/
│   ├── 03_validation_of_language_quality_features/
│   └── 04_preprocessing/

```

— 05_exploratory_and_correlation_analysis/	
— 06_model_selection/	
— 07_models/	
— 08_analysis/	
— 09_cluster_jobs/	
— results/	# Generated outputs (plots, CSVs)
— README.md	# This file
— HANDBOOK.pdf	# Step-by-step execution instructions

Datasets

File	Description
data/ full_arxiv_before_sampling.csv	Full arXiv metadata pool before stratified sampling
data/ 90k_arxiv_metadata_from_semantic_scholar.csv	90k arXiv paper dataset containing metadata and abstracts collected from Semantic Scholar
data/ 90k_arxiv_citation_prediction_full.csv	Final dataset used for citation count prediction experiments.
data/ 90k_arxiv_doi_prediction_full.csv	Final dataset used for DOI presence prediction experiments.
data/200-paper-benchmark_validation_of_linguistic_features.csv	200-paper benchmark with ground-truth labels and computed linguistic features, used for feature validation
data/ 90k_arxiv_citation_prediction_splits/	Train / validation / test splits for citation count prediction
data/ 90k_arxiv_doi_prediction_splits/	Train / validation / test splits for DOI presence prediction
data/200-paper-benchmark/	Benchmark dataset used for validating language quality measures and evaluating native-like language classification.

The prediction datasets contain the feature groups described in the thesis:

- Temporal features
- Text length and structural features
- Publication and indexing metadata
- Field and discipline features
- Author impact features
- Linguistic quality features

The exact feature definitions, preprocessing procedures, and target variable construction are described in the Methodology chapter of the thesis.

200-Paper Benchmark Files

The `data/200-paper-benchmark/` directory contains:

- Original paper PDFs
 - Extracted JSON representations
 - Plain-text paper files
 - Grammar detection outputs
 - Native-like language classification outputs
-

Results

The `results/` directory contains generated outputs used in the thesis, including:

- Exploratory analyses
 - Correlation analyses
 - Validation results
 - Model selection results
 - Ablation study results
 - Feature importance analyses
 - Figures and plots
-

Source Code

`code/01_data_collection/`

Script	Description
<code>stratified_sampler.py</code>	Draws a stratified 90k sample from the full arXiv pool, balanced by year, citation count, research field, and DOI presence
<code>clean_abstracts.py</code>	Cleans and deduplicates abstracts: removes HTML/LaTeX artifacts, filters low-quality abstracts, removes near-duplicates
<code>pdf2txt_with_fitz.py</code>	Converts a single PDF paper to a structured JSON using PyMuPDF
<code>json2txt.py</code>	Converts paper JSONs to plain-text files; strips references, repairs hyphenated line breaks

code/02_feature_extraction/

Script	Description
readability_on_arxiv_abstracts.py	Computes seven readability metrics (Flesch, Gunning Fog, Dale-Chall, etc.) on abstract text
readability_on_arxiv_papers.py	Computes the same readability metrics on full paper text via chunked aggregation
perplexity_arxiv_abstracts.py	Computes GPT-2 perplexity on abstract text using non-overlapping token windows
detect_grammar.py	Counts grammar errors in paper text using a T5-based GEC model (unbabel/gec-t5-small)
qwen_native_like_prediction.py	Assigns a native-like language score (0–10) and binary verdict to each paper using Qwen2.5-14B

code/03_validation_of_language_quality_features/

Script	Description
validation_of_language_quality_features.py	Validates that language quality features discriminate native-like from non-native-like papers using Mann-Whitney U tests and rank-biserial correlations

code/04_preprocessing/

Script	Description
preprocess.py	Merges all features, handles imputation, feature engineering, and one-hot encoding; produces datasets for citation and DOI prediction
generate_splits_citations.py	Generates stratified train/val/test splits for citation prediction
generate_splits_doi.py	Generates stratified train/val/test splits for DOI prediction

code/05_exploratory_and_correlation_analysis/

Script	Description
eda_and_correlation_analysis.py	Descriptive statistics, Mann-Whitney U group tests, Spearman correlations, and distribution plots for all feature groups

code/06_model_selection/

Script	Description
model_and_threshold_selection_for_citations.py	Evaluates three model types across seven citation thresholds on the validation set
model_selection_for_doi.py	Evaluates three model types for DOI prediction on the validation set

code/07_models/

Script	Description
citation_prediction/lightgbm_citation_prediction.py	Trains and evaluates LightGBM for citation count prediction (citation_count > 5)
citation_prediction/xgboost_citation_prediction.py	Trains and evaluates XGBoost for citation count prediction
citation_prediction/logistic_regression_citation_prediction.py	Trains and evaluates logistic regression (with p-values) for citation count prediction
doi_prediction/lightgbm_doi_prediction.py	Trains and evaluates LightGBM for DOI presence prediction
doi_prediction/xgboost_doi_prediction.py	Trains and evaluates XGBoost for DOI presence prediction
doi_prediction/logistic_regression_doi_prediction.py	Trains and evaluates logistic regression (with p-values) for DOI presence prediction

code/08_analysis/

Script	Description
ablation/ablations_for_citation_prediction.py	Runs 25 ablation configurations for citation prediction to quantify each feature group's contribution
ablation/ablations_for_doi_prediction.py	Runs 23 ablation configurations for DOI prediction
feature_importance/feature_importance_analysis_citations.py	Aggregates feature importance across XGBoost, LightGBM, and logistic regression for citation prediction
feature_importance/feature_importance_analysis_doi.py	Aggregates feature importance across models for DOI prediction

code/09_cluster_jobs/

SLURM batch scripts used to run computationally intensive feature extraction on an HPC cluster.

Not needed for local execution.

Script	Description
run_grammar_detection.sbatch	Launches <code>detect_grammar.py</code> as a 50-shard GPU array job
run_qwen_awq.sbatch	Launches <code>qwen_native_like_prediction.py</code> as a batched GPU array job

Stage 1 — Sampling (optional; 90k sample already in data/)

Draws a stratified 90k sample from the full arXiv metadata.

```
python code/01_data_collection/stratified_sampler.py
```

- **Input:** `data/full_arxiv_before_sampling.csv`
- **Output:** `data/arxiv_metadata_stratified_90k.csv`

Stage 2 — Abstract cleaning (optional; cleaned data already in data/)

Cleans and deduplicates abstracts.

```
python code/01_data_collection/clean_abstracts.py
```

- **Input:** `data/90k_arxiv_metadata_from_semantic_scholar.csv`
- **Output:**
`data/90k_arxiv_metadata_from_semantic_scholar_cleaned.csv`
- **Audit log:**
`data/90k_arxiv_metadata_from_semantic_scholar_removed.csv`

Stage 3 — PDF → JSON conversion (requires paper PDFs)

Converts a single PDF to JSON. Paper PDFs are **not included** in this submission.

```
python code/01_data_collection/pdf2txt_with_fitz.py \  
--input PATH/T0/paper.pdf \  
--output PATH/T0/output_dir/
```

Stage 4 — JSON → TXT conversion (requires paper JSONs from Stage 3)

Converts paper JSONs to plain-text files. Run one of the following modes:

```
# Single file
python code/01_data_collection/json2txt.py \
  --in_json PATH/T0/paper.json \
  --out_dir PATH/T0/txt_output/

# Whole directory
python code/01_data_collection/json2txt.py \
  --in_dir PATH/T0/json_dir/ \
  --out_dir PATH/T0/txt_output/

# From a list file (recommended for large batches)
python code/01_data_collection/json2txt.py \
  --list_file PATH/T0/file_list.txt \
  --out_dir PATH/T0/txt_output/
```

Stage 5 — Feature extraction on abstracts (optional; features already in full CSVs)

Readability metrics (abstracts)

Runs as 50 shards via SLURM_ARRAY_TASK_ID; defaults to shard 0 locally.

```
python code/02_feature_extraction/readability_on_arxiv_abstracts.py
```

- **Input:** data/90k_arxiv_metadata_from_semantic_scholar.csv
- **Output:**
data/abstract_readability_50_shards/abstract_readability_shard_00.csv (and one file per shard)

Perplexity (abstracts)

Runs as 50 shards; defaults to shard 0 locally. Requires a GPU for reasonable speed.

```
python code/02_feature_extraction/perplexity_arxiv_abstracts.py \
  --array_index 0 \
  --num_shards 1
```

- **Input:** data/90k_arxiv_metadata_from_semantic_scholar.csv
 - **Output:** data/abstract_gpt2_ppl_outputs_50shards/
-

Stage 6 — Feature extraction on full papers (requires paper TXTs from Stage 4)

Paper TXT files are **not included** in this submission. Pass the directory containing them via CLI arguments.

Readability metrics (papers)

```
python code/02_feature_extraction/readability_on_arxiv_papers.py \
  --input_dir PATH/T0/txt_dir/ \
  --output_csv results/paper_readability.csv \
  --split train # or test
```

Grammar error detection

```
python code/02_feature_extraction/detect_grammar.py \
  --input_root PATH/T0/txt_root/ \
  --output_root results/grammar_outputs/ \
  --shard_id 0 \
  --n_shards 1
```

Or for a single file:

```
python code/02_feature_extraction/detect_grammar.py \
  --input PATH/T0/paper.txt \
  --output results/grammar_outputs/paper.json
```

Native-likeness prediction (GPU required)

```
python code/02_feature_extraction/qwen_native_like_prediction.py \
  --in_dir PATH/T0/txt_dir/ \
  --out_dir results/qwen_outputs/ \
  --model_path Qwen/Qwen2.5-14B-Instruct-AWQ
```

The 09_cluster_jobs/ sbatch scripts show how these were launched on a SLURM cluster.

Stage 7 — Validation of language quality features

Runs Mann-Whitney U tests and rank-biserial correlations on the 200-paper benchmark.

```
python
code/03_validation_of_language_quality_features/validation_of_language_quality_f
eatures.py
```

- **Input:** data/200-paper-benchmark_validation_of_linguistic_features.csv
 - **Output:** data/200-paper-benchmark/linguistic_feature_boxplots/ (plots + CSV)
-

Stage 8 — Preprocessing (optional; preprocessed data already in data/)

Merges and encodes all features into classifier-ready datasets.

```
python code/04_preprocessing/preprocess.py
```

- **Input:** data/90k_arxiv_citation_prediction_full.csv
 - **Output:** data/preprocessed_90k_classifiers/
-

Stage 9 — Generate train/val/test splits (optional; splits already in data/)

```
python code/04_preprocessing/generate_splits_citations.py
python code/04_preprocessing/generate_splits_doi.py
```

- **Inputs:** data/90k_arxiv_citation_prediction_full.csv, data/90k_arxiv_doi_prediction_full.csv
 - **Outputs:** data/90k_arxiv_citation_prediction_splits/, data/90k_arxiv_doi_prediction_splits/
-

Stage 10 — Exploratory data analysis

```
python
code/05_exploratory_and_correlation_analysis/eda_and_correlation_analysis.py
```

- **Inputs:** data/90k_arxiv_citation_prediction_full.csv, data/90k_arxiv_doi_prediction_full.csv
 - **Output:** results/eda_publication_success_outputs/
-

Stage 11 — Model and threshold selection

```
python code/06_model_selection/model_and_threshold_selection_for_citations.py
python code/06_model_selection/model_selection_for_doi.py
```

- **Inputs:** split CSVs from data/90k_arxiv_citation_prediction_splits/ and data/90k_arxiv_doi_prediction_splits/
 - **Output:** results/model_selection/
-

Stage 12 — Model training and evaluation

Each script trains the model and prints evaluation metrics to stdout.

Citation count prediction (citation_count > 5)

```
python code/07_models/citation_prediction/lightgbm_citation_prediction.py
python code/07_models/citation_prediction/xgboost_citation_prediction.py
python
code/07_models/citation_prediction/logistic_regression_citation_prediction.py
```

DOI presence prediction

```
python code/07_models/doi_prediction/lightgbm_doi_prediction.py
python code/07_models/doi_prediction/xgboost_doi_prediction.py
python code/07_models/doi_prediction/logistic_regression_doi_prediction.py
```

- **Inputs:** split CSVs from data/90k_arxiv_citation_prediction_splits/ and data/90k_arxiv_doi_prediction_splits/
-

Stage 13 — Ablation study

```
python code/08_analysis/ablation/ablations_for_citation_prediction.py
python code/08_analysis/ablation/ablations_for_doi_prediction.py
```

- **Output:** ablation_citation_gt_5_results_xgboost.csv, ablation_doi_prediction_results_xgboost.csv (written to the working directory)
-

Stage 14 — Feature importance analysis

```
python
code/08_analysis/feature_importance/feature_importance_analysis_citations.py
python code/08_analysis/feature_importance/feature_importance_analysis_doi.py
```

- **Output:** results/feature_importance_plots/
-

Notes

- **Stages 1–2, 8–9** are optional because the processed data files are already included in data/.
- **Stages 3–4, 6** require paper PDFs or TXTs which are not included in this submission. All scripts in those stages accept input paths as command-line arguments.
- **Stage 6 (Qwen)** additionally requires a GPU and either a local copy of Qwen2.5-14B-Instruct-AWQ or a HuggingFace download.
- The 09_cluster_jobs/ folder contains SLURM batch scripts used to run grammar detection and native-like language classification on an HPC cluster. They are provided for reference only and are not needed for local execution.